

Micro Hack 2 - SDC Workloads Workbook

FY27 - Fabric Apps Motion (EMEA)

Contents

1 Micro Hack 2 — SDC Workloads Workbook (Participants)	1
1.1 Day agenda (1 day)	1
2 Part A — Understand	2
2.1 1) The scenario in plain words	2
2.2 2) The three pieces (and who owns the algorithm)	4
2.3 3) Key words (30-second primer)	4
2.4 4) Before you start — checklist	4
2.5 5) Team framing (fill this in 2 minutes, then build)	4
3 Part B — Build it step by step (full solution)	4
3.1 Step 1 — Clone the toolkit	5
3.2 Step 2 — Configure the dev environment (one script)	5
3.3 Step 3 — Start the dev server + the Dev Gateway	5
3.4 Step 4 — Hello World test (prove the gateway works, before building anything)	6
3.5 Step 5 — Create the GreenGrid item	6
3.6 Step 6 — Check the SaaS is alive	6
3.7 Step 7 — Make it work with fake data (Milestone M1)	6
3.8 Step 8 — Switch to real OneLake data (Milestone M2)	9
3.9 Step 9 — Make it graphical (Milestone M3)	11
4 Wrap-up	11
4.1 Demo storyboard (5 minutes)	11
4.2 Reflection	11
4.3 Reference assets	11

1 Micro Hack 2 — SDC Workloads Workbook (Participants)

Track: **SDC Workloads (Fabric Extensibility Toolkit)** Scenario: **GreenGrid Analytics** (fictional SDC)

Print-ready PDF: [microhack-2-sdc-workloads-workbook.pdf](#)

This is a **didactic** micro-hack. You are **not** expected to *find* the solution — every step tells you **exactly what to type or paste, why, and what you should see**. Follow them in order and you get a working Fabric workload, starting from a clean machine.

1.1 Day agenda (1 day)

Morning — presentations & demos. Afternoon — hands-on hack. This is a **standalone one-day micro-hack** for the SDC Workloads (Extensibility Toolkit) track.

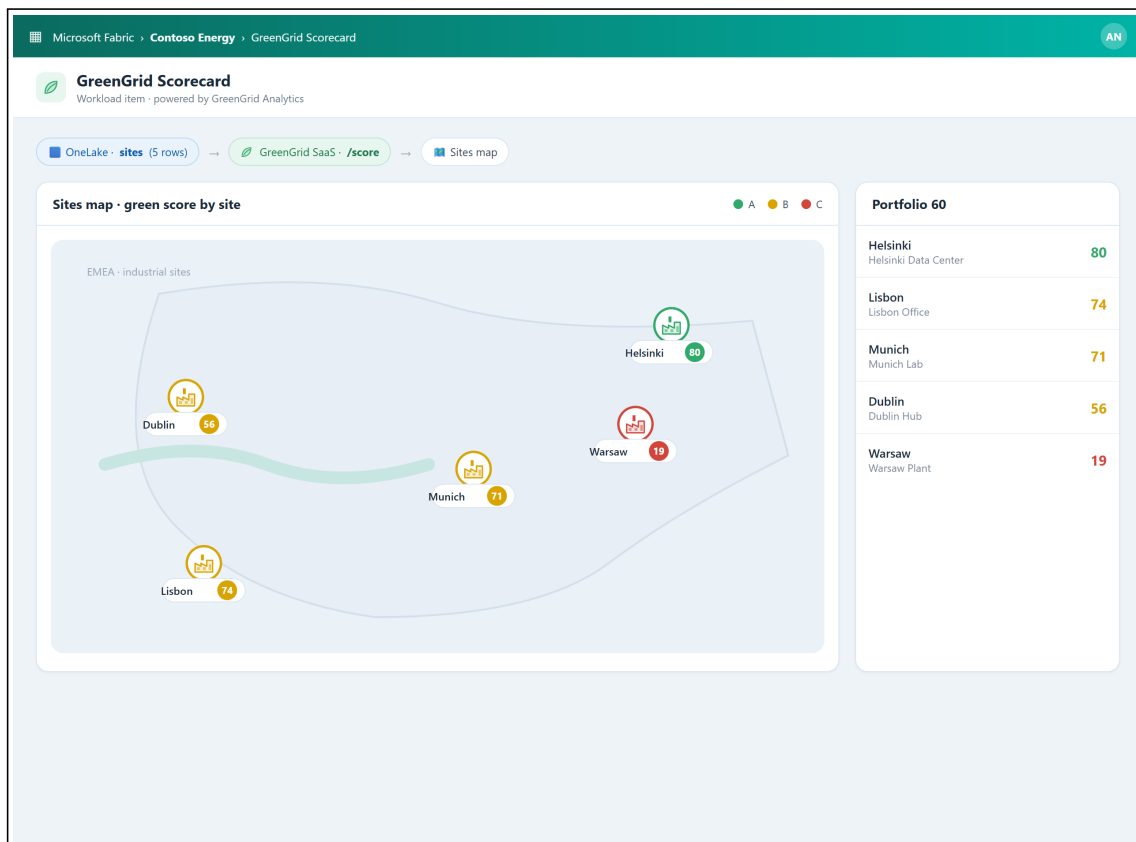
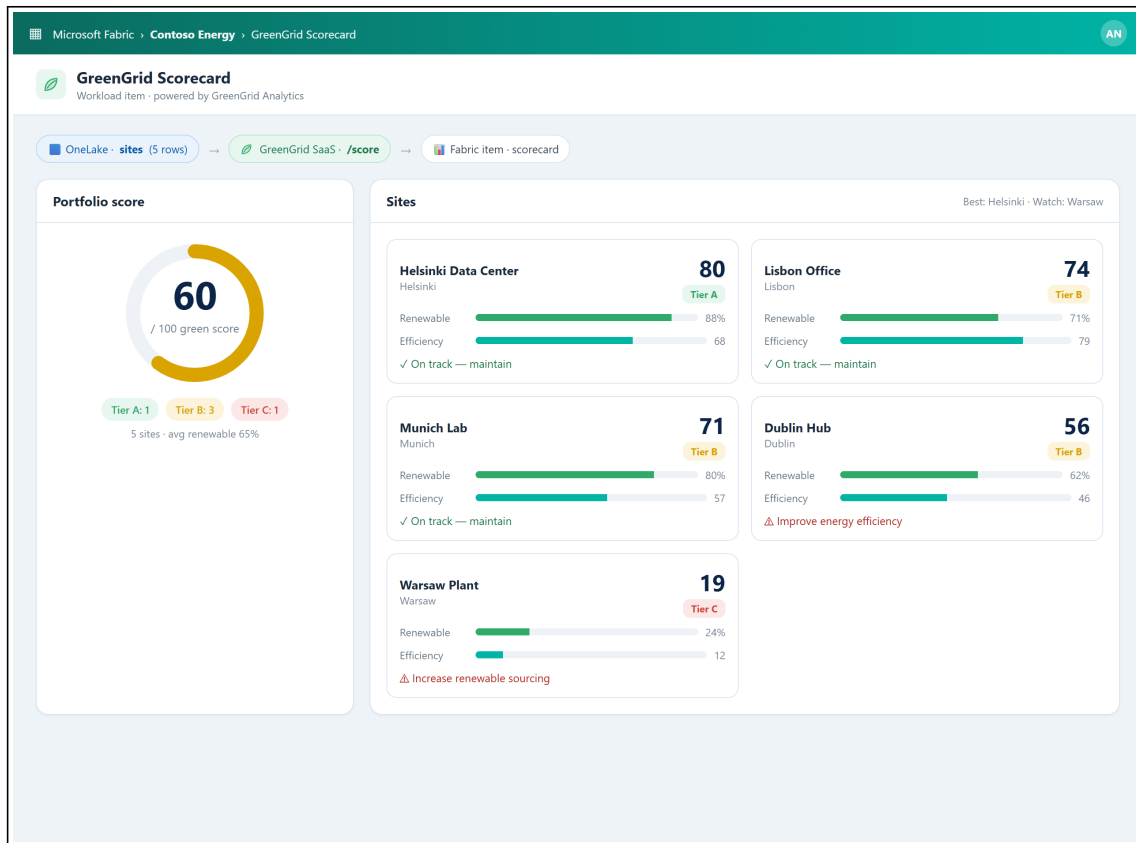
Time	Session
09:00	Welcome & motion overview — the Fabric apps motion for EMEA
09:30	Microsoft Fabric foundations — OneLake, capacity, workspaces
10:00	Extensibility Toolkit foundations — workloads, items, Dev Gateway
10:30	Break
10:45	Demo · GreenGrid Scorecard — the workload you'll build + the SaaS prerequisite
11:15	The hack brief, teams & environment check
12:00	Lunch
13:30	Hack · Sprint 1 — clone, run Hello World, render with fake data (steps 1–7)
15:30	Break
15:45	Hack · Sprint 2 — switch to OneLake & polish (steps 8–9)
16:45	Team demos (5 min each)
17:00	Wrap-up, KPIs & next steps

2 Part A — Understand

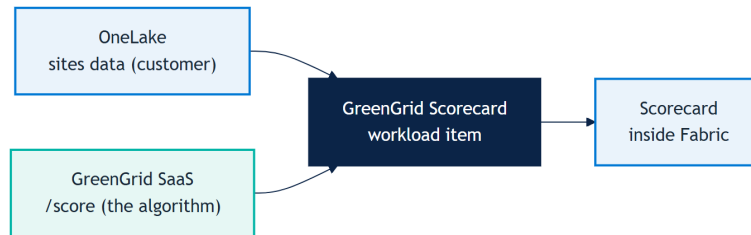
2.1 1) The scenario in plain words

GreenGrid Analytics is a software company (an **SDC** — a solution development company). Its product scores how **sustainable** a customer's sites are, from their energy use and renewable mix. The customer's data already lives in Microsoft Fabric (**OneLake**). Your job: bring GreenGrid's scoring **inside Fabric**, as a native screen, **without moving the data**.

The result looks like this — a sustainability scorecard and a map of the industrial sites, rendered inside Fabric:



2.2 2) The three pieces (and who owns the algorithm)



- **OneLake** = where the **customer's data** lives.
- **GreenGrid SaaS** = a small web service that **holds the scoring algorithm** (GreenGrid's IP). *The trainer deploys it for you — it is a prerequisite.*
- **The workload** = the piece **you build today**: a Fabric *item* that reads OneLake, calls the SaaS, and draws the scorecard.

2.3 3) Key words (30-second primer)

- **Workload**: a web app that runs *inside* the Fabric portal. You host it, Fabric shows it.
- **Item**: one thing a user creates in a workspace (here: a *GreenGrid Scorecard*).
- **Extensibility Toolkit**: Microsoft's open-source kit + SDK to build such workloads.
- **Dev Gateway**: a local bridge so Fabric can show your workload while it runs on **your machine** — this is what makes a one-day hack possible.
- **OBO token** (On-Behalf-Of): a secure token the workload gets so it can read the user's OneLake data **as that user** — no passwords, no data copies.

2.4 4) Before you start — checklist

- ☐ **Node.js, PowerShell 7, .NET (x64), Azure CLI, VS Code** installed.
- ☐ A **Fabric workspace** with a **capacity** assigned (from the trainer).
- ☐ Permission to create an **Entra app** (or one provided by the trainer).
- ☐ The **GreenGrid SaaS URL + API key** (from the trainer).

2.5 5) Team framing (fill this in 2 minutes, then build)

- Team name:
 - SaaS URL + API key:
 - One sentence you'll say in the demo:
-

3 Part B — Build it step by step (full solution)

Golden rule (read this!): first get the toolkit running with the built-in **Hello World** (steps 1–4). Only then build GreenGrid (steps 5–9). And inside GreenGrid, render **fake data first** (step 7), *then* switch to real OneLake data (step 8). Never debug security before something shows on screen.

Every step says **what it is**, **what to type/paste**, and **what you should see**.

3.1 Step 1 — Clone the toolkit

What this is. Get Microsoft's Extensibility Toolkit on your machine. This is a real repository — you clone it and start from it.

```
git clone https://github.com/microsoft/fabric-extensibility-toolkit
cd fabric-extensibility-toolkit
```

What you should see. A fabric-extensibility-toolkit folder with Workload/ and scripts/ inside.

3.2 Step 2 — Configure the dev environment (one script)

What this is. A setup script that creates the Entra app, writes your .env, and downloads the Dev Gateway. It asks you to sign in and pick your workspace.

```
cd scripts/Setup
./Setup.ps1 -WorkloadName "Org.GreenGrid"
```

The WorkloadName must look like Organization.Name — use Org.GreenGrid for the hack. On macOS/Linux run pwsh ./Setup.ps1 -WorkloadName "Org.GreenGrid".

What you should see. The script ends with “setup complete” and prints the next steps.

3.3 Step 3 — Start the dev server + the Dev Gateway

What this is. Two long-running processes, in **two terminals**. One serves your workload UI from localhost; the other connects Fabric to that localhost.

Terminal 1 – the frontend + APIs

```
cd scripts/Run
./StartDevServer.ps1
```

Terminal 2 – the bridge to Fabric

```
cd scripts/Run
./StartDevGateway.ps1
```

Then, in the Fabric portal: open **Settings -> Admin portal** and enable the developer tenant settings, then **Settings -> Developer settings -> enable Fabric Developer Mode**.

What you should see. Both terminals keep running without errors, and Developer Mode is ON in Fabric.

3.4 Step 4 — Hello World test (prove the gateway works, before building anything)

What this is. The toolkit ships a **Hello World** item. Create it in Fabric to confirm the whole chain — *your localhost code -> Dev Gateway -> Fabric* — works. **This is your safety checkpoint.**

1. Go to <https://app.fabric.microsoft.com/workloadhub/detail/Org.GreenGrid.Product?exp=developer> (the workload shows as “Hello Fabric!” until you rename it).
2. Click the **Hello World** item type on the left.
3. Pick your **dev workspace** in the dialog.
4. The Hello World editor opens.

What you should see. The Hello World editor renders **inside Fabric**, served from your machine. If you see it, everything works — now you can build GreenGrid. (*Nothing shows up? Developer Mode is off, or one of the two terminals isn't running.*)

3.5 Step 5 — Create the GreenGrid item

What this is. Generate a fresh item to hold your scorecard (don't overwrite Hello World).

```
cd scripts/Setup
./CreateNewItem.ps1 -Name "GreenGridScorecard"
```

This creates `Workload/app/items/GreenGridScorecardItem/`, including `GreenGridScorecardItemEditor.tsx`. **Restart the dev server** (Ctrl+C in Terminal 1, then `./StartDevServer.ps1` again). In Fabric, create a **GreenGridScorecard** item.

What you should see. An empty `GreenGridScorecard` editor opens in Fabric.

3.6 Step 6 — Check the SaaS is alive

What this is. A quick test that GreenGrid's algorithm service answers.

```
curl -i <SAAS_URL>/health # expect: 200
curl -s -X POST <SAAS_URL>/score \
  -H "Content-Type: application/json" -H "x-api-key: <API_KEY>" \
  -d '{"sites":[{"siteId":"s1","name":"Helsinki",
    ↪ "DC","city":"Helsinki","energyKwh":320,"renewablePct":88}]}'
```

What you should see. `/score` returns a `greenScore` and a `tier`. (*Local fallback: `cd src/workloadsrc && npm install && npm run saas:start`, URL `http://localhost:8787`, key `greengrid-demo-key`.*)

3.7 Step 7 — Make it work with fake data (Milestone M1)

Create **four files** in `Workload/app/items/GreenGridScorecardItem/`, then render the scorecard from the generated editor. This proves the whole chain *screen -> SaaS -> screen*.

3.7.1 7a. contracts.ts — the data shapes

```
export type SiteRecord = {
  siteId: string; name: string; city: string; energyKwh: number;
  ↪ renewablePct: number;
};
export type GreenTier = 'A' | 'B' | 'C';
export type ScoredSite = SiteRecord & {
  efficiency: number; greenScore: number; tier: GreenTier; tip: string;
};
export type ScoreResponse = {
  sites: ScoredSite[];
  summary: { totalSites: number; avgScore: number; avgRenewablePct: number;
    tierCounts: Record<GreenTier, number>; best: ScoredSite; worst:
  ↪ ScoredSite; };
};
```

3.7.2 7b. greengridClient.ts — call the SaaS

```
import type { ScoreResponse, SiteRecord } from './contracts';

const SAAS_URL = 'http://localhost:8787'; // <- replace with the trainer
  ↪ URL
const API_KEY = 'greengrid-demo-key'; // <- replace with the trainer
  ↪ key

export async function scorePortfolio(sites: SiteRecord[]):
  ↪ Promise<ScoreResponse> {
  const res = await fetch(`${SAAS_URL}/score`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', 'x-api-key': API_KEY },
    body: JSON.stringify({ sites }),
  });
  if (!res.ok) throw new Error(`GreenGrid SaaS error ${res.status}`);
  return res.json();
}
```

3.7.3 7c. seed.ts — fake sites to start

```
import type { SiteRecord } from './contracts';
export const seedSites: SiteRecord[] = [
  { siteId: 'site-hel', name: 'Helsinki Data Center', city: 'Helsinki',
  ↪ energyKwh: 320, renewablePct: 88 },
  { siteId: 'site-lis', name: 'Lisbon Office', city: 'Lisbon', energyKwh:
  ↪ 210, renewablePct: 71 },
  { siteId: 'site-muc', name: 'Munich Lab', city: 'Munich', energyKwh:
  ↪ 430, renewablePct: 80 },
```



```

    { siteId: 'site-dub', name: 'Dublin Hub', city: 'Dublin', energyKwh:
    ↪ 540, renewablePct: 62 },
    { siteId: 'site-waw', name: 'Warsaw Plant', city: 'Warsaw', energyKwh:
    ↪ 880, renewablePct: 24 },
  ];

```

3.7.4 7d. Scorecard.tsx — the screen

It takes a `getSites` function, so swapping the data source later is **one line**.

```

import React, { useEffect, useState } from 'react';
import { FluentProvider, webLightTheme, Spinner, Badge } from
    ↪ '@fluentui/react-components';
import { scorePortfolio } from './greengridClient';
import type { ScoreResponse, SiteRecord } from './contracts';

const tierColor = (t: 'A' | 'B' | 'C') => (t === 'A' ? '#2faa6a' : t ===
    ↪ 'B' ? '#d9a300' : '#d1453b');

export function Scorecard({ getSites }: { getSites: () =>
    ↪ Promise<SiteRecord[]> }) {
  const [data, setData] = useState<ScoreResponse | null>(null);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    getSites().then(scorePortfolio).then(setData).catch((e) =>
    ↪ setError(String(e)));
  }, [getSites]);

  if (error) return <div role="alert">Failed: {error}</div>;
  if (!data) return <Spinner label="Scoring sites..." />;

  return (
    <FluentProvider theme={webLightTheme}>
      <div style={{ padding: 24 }}>
        <h1>GreenGrid Scorecard</h1>
        <p>Data from OneLake, scored by GreenGrid</p>
        <div style={{ fontSize: 48, fontWeight: 700
    ↪ }}>{data.summary.avgScore}<span style={{ fontSize: 18 }}> /
    ↪ 100</span></div>
        <div style={{ display: 'grid', gridTemplateColumns:
    ↪ 'repeat(2,1fr)', gap: 12, marginTop: 16 }}>
          {data.sites.map((s) => (
            <div key={s.siteId} style={{ border: '1px solid #dbe6f1',
    ↪ borderRadius: 12, padding: 14 }}>
              <div style={{ display: 'flex', justifyContent:
    ↪ 'space-between' }}>
                <strong>{s.name}</strong>

```

```

        <Badge appearance="filled" style={{ background:
↪ tierColor(s.tier) }}>Tier {s.tier}</Badge>
        </div>
        <div style={{ color: '#7a8a9a', fontSize: 12 }}>{s.city}</div>
        <div style={{ fontSize: 26, fontWeight: 700
↪ }}>{s.greenScore}</div>
        <div style={{ fontSize: 13 }}>{s.tip}</div>
        </div>
    ))}
  </div>
</div>
</FluentProvider>
);
}

```

3.7.5 7e. Render it from the generated editor

Open `GreenGridScorecardItemEditor.tsx` and render the Scorecard with the **fake** data:

```

import { Scorecard } from './Scorecard';
import { seedSites } from './seed';

// inside the editor's returned JSX:
<Scorecard getSites={() => Promise.resolve(seedSites)} />

```

What you should see — Milestone M1. The item shows the portfolio score and 5 site cards with their tier, **inside Fabric**. The whole chain works on fake data. This is demoable.

3.8 Step 8 — Switch to real OneLake data (Milestone M2)

What this is. Replace the fake `seedSites` with the customer's **real** site data in OneLake.

3.8.1 Where does the data live?

In Fabric, customer data lives in a **Lakehouse** inside OneLake. A Lakehouse has **Files** (raw files: CSV, Parquet...) and **Tables** (Delta). For this hack the sites are a **CSV file**: `Files/sites.csv`. *(A CSV is the simplest thing to read from a workload; a Delta table would need a SQL endpoint — out of scope for one day.)*

Format of `Files/sites.csv` (a ready copy is in the kit at `src/workloadsdc/data/sites.csv`):

```

siteId,name,city,energyKwh,renewablePct
site-hel,Helsinki Data Center,Helsinki,320,88
site-lis,Lisbon Office,Lisbon,210,71
site-muc,Munich Lab,Munich,430,80
site-dub,Dublin Hub,Dublin,540,62
site-waw,Warsaw Plant,Warsaw,880,24

```

The trainer pre-loads this file (Lakehouse -> **Files** -> **Upload**). To test your own, upload a CSV with the **same header** to Files/.

You also need two IDs (visible in the Lakehouse URL, or from the item context): `workspaceId` and `lakehouseId`.

3.8.2 8a. onelake.ts — read & parse the CSV

```
import type { WorkloadClientAPI } from '@ms-fabric/workload-client';
import type { SiteRecord } from './contracts';

export async function readSites(
  client: WorkloadClientAPI, workspaceId: string, lakehouseId: string
): Promise<SiteRecord[]> {
  // 1) Token to read OneLake AS the signed-in user (OBO).
  const { token } = await client.auth.acquireAccessToken({
    additionalScopesToConsent:
    ↪ ['https://storage.azure.com/user_impersonation'],
  });
  // 2) Read the CSV file from the Lakehouse "Files" area.
  const url = `https://onelake.dfs.fabric.microsoft.com/${workspaceId}/${lakehouseId}/Files/sites.csv`;
  ↪ lakehouseId}/Files/sites.csv`;
  const res = await fetch(url, { headers: { Authorization: `Bearer ${token}` } });
  ↪ `${token}` } });
  if (!res.ok) throw new Error(`OneLake read failed: ${res.status}`);
  // 3) Parse CSV text into rows.
  const [header, ...lines] = (await res.text()).trim().split(/\r?\n/);
  const cols = header.split(',').map((c) => c.trim());
  return lines.filter(Boolean).map((line) => {
    const cells = line.split(',');
    const row = Object.fromEntries(cols.map((c, i) => [c, (cells[i] ??
    ↪ '').trim()]));
    return { siteId: row.siteId, name: row.name, city: row.city,
      energyKwh: Number(row.energyKwh), renewablePct:
    ↪ Number(row.renewablePct) };
  });
}
```

3.8.3 8b. Swap one line in the editor

The generated editor exposes the workload client and the item's context. Change the render to:

```
import { readSites } from './onelake';

// before (fake):
<Scorecard getSites={() => Promise.resolve(seedSites)} />

// after (real OneLake CSV):
```

```
<Scorecard getSites={() => readSites(workloadClient, workspaceId,
  ↪ lakehouseId)} />
```

What you should see — Milestone M2. The same scorecard, now built from the **OneLake CSV**. *(If the OBO token fights you, keep the fake-data line as a fallback and keep moving.)*

3.9 Step 9 — Make it graphical (Milestone M3)

What this is. Upgrade the plain list into the polished scorecard + map. Easiest with GitHub Copilot (the toolkit ships AI instructions in `.github/copilot-instructions.md`):

Upgrade the GreenGrid Scorecard screen: add a circular green-score gauge
 ↪ for the portfolio,
 an A/B/C tier distribution, provenance chips "Data from OneLake" then
 ↪ "Scored by GreenGrid",
 and a second view showing the sites as factory markers on a simple map,
 ↪ colored by tier.
 Style: Fluent UI, teal #00b4a6 + green accents. Keep the `getSites` data
 ↪ flow.

What you should see — Milestone M3. A gauge, A/B/C tiers and the sites map — matching the example images at the top.

4 Wrap-up

4.1 Demo storyboard (5 minutes)

1. **Context** (30s): the SaaS value (the algorithm) you brought into Fabric.
2. **Flow** (3m): open the item, read OneLake, call the SaaS, show the scorecard + map.
3. **Architecture** (45s): OneLake (data) + SaaS (algorithm) + workload (native UI).
4. **Publish plan** (45s): from Dev Gateway to a released workload.

4.2 Reflection

- Where did the OneLake -> SaaS contract surprise you?
- What stays in the SaaS (the IP) vs. in the workload (the experience)?
- What would you harden before production?

4.3 Reference assets

- Full solution code: `src/workloadscdc/`
- SaaS prerequisite: `src/workloadscdc/saas/`
- Sample OneLake data: `src/workloadscdc/data/sites.csv`
- Trainer answer key + screenshots: `docs/microhack-2-sdc-workloads-solution.md`